



MODULE 5

HYPERLEDGER

Hyperledger is not a blockchain, but it is a project that was initiated by the Linux Foundation in December 2015 to advance blockchain technology. This project is a collaborative effort by its members to build an open-source distributed ledger framework that can be used to develop and implement cross-industry blockchain applications and systems. The principal focus is to develop and run platforms that support global business transactions. The project also focuses on improving the reliability and performance of blockchain systems.

Projects under Hyperledger undergo various stages of development, starting from proposal to incubation and graduating to an active state. Projects can also be deprecated or in end-of-life state where they are no longer actively developed. For a project to be able to move into the incubation stage, it must have a fully working code base along with an active community of developers.

Projects under Hyperledger

There are two categories of projects under Hyperledger. The first is blockchain projects and the second category is relevant tools or modules that support these blockchains. Currently, there are five blockchain framework projects under the Hyperledger umbrella: Fabric, Sawtooth Lake, Iroha, Burrow, and Indy. Under modules, there are the Hyperledger Cello, Hyperledger Composer,

Hyperledger Explorer, and Hyperledger Quilt. The Hyperledger project currently has more than 200-member organizations and is very active with many contributors, with regular meet-ups and talks organized around the globe.

A brief introduction of all these projects follows, after which we will see more details around the design, architecture, and implementation of Fabric and Sawtooth Lake.

Hyperledger as a protocol

Hyperledger is aiming to build new blockchain platforms that are driven by industry use cases. As there have been many contributions made to the Hyperledger project by the community, Hyperledger blockchain platform is evolving into a protocol for business transactions. Hyperledger is also evolving into a specification that can be used as a reference to build blockchain platforms as compared to earlier blockchain solutions that address only a specific type of industry or requirement.



In the following section, a reference architecture is presented that has been published by the Hyperledger project. As this work is under continuous and rigorous development, some changes are expected in this, but core services are expected to remain unchanged.

Fabric

To understand various projects that are under development in the Hyperledger project, it is essential to understand the foundations of Hyperledger first. A few terminologies that are specific to Hyperledger need some clarification before readers are introduced to the more in-depth material.

First, there is the concept of fabric. Fabric can be defined as a collection of components providing a foundation layer that can be used to deliver a blockchain network. There are various types and capabilities of a fabric network, but all fabrics share common attributes such as immutability and are consensus-driven. Some fabrics can provide a modular approach towards building blockchain networks. In this case, the blockchain network can have multiple pluggable modules to perform a various function on the network.

For example, consensus algorithms can be a pluggable module in a blockchain network where, depending on the requirements of the network, an appropriate consensus algorithm can be chosen and plugged into the network. The modules can be based on some particular specification of the fabric and can include APIs, access control, and various other components.

Fabrics can also be designed either to be private or public and can allow the creation of multiple business networks. As an example, Bitcoin is an application that runs on top of its fabric (blockchain network). As discussed earlier in Chapter 1, Blockchain 101, blockchain can either be permissioned or permissionless. However, the aim of Hyperledger Fabric is to develop a permissioned distributed ledger.

Fabric is also the name given to the code contribution made by IBM to the Hyperledger foundation and is formally called Hyperledger Fabric. IBM also offers blockchain as a service (IBM Blockchain) via its IBM Cloud service.



HYPERLEDGER FABRIC

The fabric is the contribution made initially by IBM and Digital Assets to the Hyperledger project. This contribution aims to enable a modular, open, and flexible approach towards building blockchain networks.

Various functions in the fabric are pluggable, and it also allows the use of any language to develop smart

contracts. This functionality is possible because it is based on container technology (Docker), which can host any language.

Chaincode is sandboxed in a secure container, which includes a secure operating system, chaincode language, runtime environment, and SDKs for Go, Java, and Node.js. Other languages can be supported too in future, if required, but needs some development work. Smart contracts are called chaincode in the fabric. This ability is a compelling feature compared to domain-specific languages in Ethereum, or the limited scripted language in Bitcoin. It is a permissioned network that aims to address issues such as scalability, privacy, and confidentiality. The fundamental idea behind this is modularization, which would allow for flexibility in design and implementation of the business blockchain. This can then result in achieving scalability, privacy, and other desired attributes and fine tune them according to the requirements.

Transactions in the fabric are private, confidential, and anonymous for general users, but they can still be traced

and linked to the users by authorized auditors. As a permissioned network, all participants are required to be registered with the membership services to access the blockchain network. This ledger also provided auditability functionality to meet the regulatory and compliance needs required by the user.

Membership services

These services are used to provide access control capability for the users of the fabric network. The following list shows the functions that membership services perform:

- User identity verification
- User registration
- Assign appropriate permissions to the users depending on their roles

Membership services make use of a certificate authority in order to support identity management and authorization operations. This CA can be internal (Fabric CA), which is a default interface in Hyperledger Fabric or **organization**



can opt to use an external certificate authority. Fabric CA issues enrollment certificates (E-Certs), which are produced by enrollment certificate authority (E-CA). Once peers are issued with an identity, they are allowed to join the blockchain network. There are also temporary certificates issued called TCerts, which are used for one-time transactions.

All peers and applications are identified using certificate authority. Authentication service is provided by the certificate authority. MSPs can also interface with existing identity services like LDAP.

Consensus services

A consensus service is responsible for providing the interface to the consensus mechanism. This serves as a module that is pluggable and receives the transaction from other Hyperledger entities and executes them under criteria according to the type of mechanism chosen.

Consensus in Hyperledger V1 is implemented as a peer called orderer, which is responsible for ordering the transactions in sequence into a block. Orderer does not hold smart contracts or ledgers. Consensus is pluggable and currently, there are two types of ordering services available in Hyperledger Fabric:

SOLO: This is a basic ordering service intended to be used for development and testing purposes.

Kafka: This is an implementation of Apache Kafka, which provides ordering service. It should be noted that currently Kafka only provides crash fault tolerance but does not provide byzantine fault tolerance. This is acceptable in a permissioned network where chances of malicious actors are almost none.

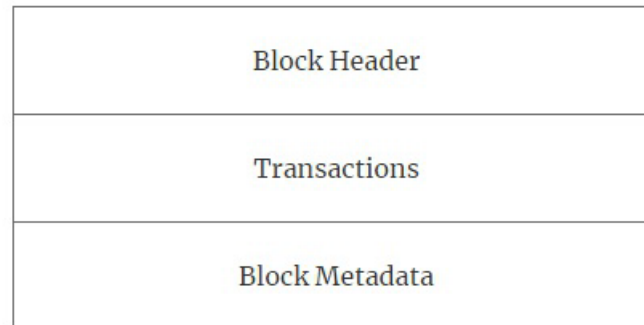
In addition to these mechanisms, the Simple Byzantine Fault Tolerance (SBFT) based mechanism is also under development, which will become available in the later releases of Hyperledger Fabric.

Distributed ledger

Blockchain and world state are two main elements of the distributed ledger. Blockchain is simply a cryptographically linked list of blocks (as introduced in Chapter 1, Blockchain 101) and world state is a key-value database. This database is used by smart contracts to store relevant states during execution by the transactions. The blockchain consists of blocks that contain transactions. These transactions contain chaincode, which runs transactions that can result in updating the world state. Each node saves the world state on disk in LevelDB or CouchDB depending on the implementation. As Fabric allows pluggable data store, you can choose any data store for storage.

A block consists of three main components called Block header, Transactions (Data) and block metadata.

The following diagram shows a typical block in the Hyperledger Fabric 1.0 with the relevant fields:



Block Header consists of three fields, namely Number, Previous hash, and Data hash.

Transaction is made up of multiple fields such as transaction type, version, timestamp, channel ID, transaction ID, epoch, payload visibility, chaincode path, chaincode name, chaincode version, creator identity, signature, chaincode type, input, timeout, endorser identities and signatures, proposal hash, chaincode events, response status, namespace, read set, write set, start key, end key, list of read, and Merkle tree query summary.

Block Metadata consists of creator identity, relevant signatures, last configuration block number, flag for each transaction included in the block, and last offset persisted (kafka).

The peer to peer protocol

The P2P protocol in the Hyperledger Fabric is built using google RPC (gRPC). It uses protocol buffers to define the structure of the messages.

Messages are passed between nodes in order to perform various functions. There are four main types of messages in Hyperledger Fabric: discovery, transaction, synchronization, and consensus. Discovery messages are exchanged between nodes when starting up in order to discover other peers on the network. Transaction messages are used to deploy, invoke, and query transactions, and consensus messages are exchanged during consensus. Synchronization messages are passed between nodes to synchronize and keep the blockchain updated on all nodes.

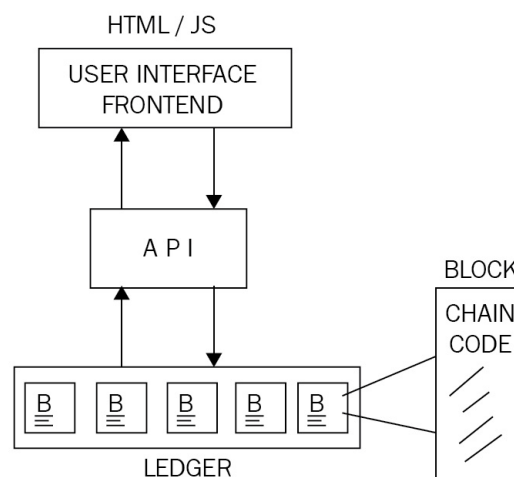
Ledger storage

In order to save the state of the ledger, by default, LevelDB is used which is available at each peer. An

alternative is to use CouchDB which provides the ability to run rich queries.

Chaincode services

These services allow the creation of secure containers that are used to execute the chaincode. Components in this category are as follows:



Secure container: Chaincode is deployed in Docker containers that provide a locked down sandboxed environment for smart contract execution. Currently, Golang is supported as the main smart contract language, but any other mainstream languages can be added and enabled if required.

Secure registry: This provides a record of all images containing smart contracts.

Events

Events on the blockchain can be triggered by endorsers and smart contracts. External applications can listen to these events and react to them if required via event adapters. They are similar to the concept of events introduced in solidity in Chapter 14, Introducing Web3.

APIs and CLIs



An application programming interface provides an interface into the fabric by exposing various REST APIs. Additionally, command-line interfaces that provide a subset of REST APIs and allow for quick testing and limited interaction with the blockchain are also available.

Components of the fabric

There are various components that can be part of the Hyperledger Fabric blockchain. These components include but are not limited to the ledger, chaincode, consensus mechanism, access control, events, system monitoring and management, wallets, and system integration components.

Peers

Peers participate in maintaining the state of the distributed ledger. They also hold a local copy of the distributed ledger. Peers communicate via gossip protocol. There are three types of peers in the Hyperledger Fabric network:

Endorsing peers or endorsers which simulate the transaction execution and generate a read-write set. Read is a simulation of transaction's reading of data from the ledger and write is the set of updates that would be made to the ledger if and when the transaction is executed and committed to the ledger. Endorsers execute and endorse transactions. It should be noted that an endorser is also a committer too. Endorsement policies are implemented with chaincode and specify the rules for transaction endorsement.

Committing peers or committers which receives transaction endorsed by endorsers, verify them and then update the ledger with the read-write set. A committer verifies the read-write set generated by the endorsers along with transaction validation. Submitters is the third type of peers which has not been implemented yet. It is on the development roadmap and will be implemented

Orderer nodes

Ordering nodes receive transactions from endorsers along with read-write sets, arrange them in a sequence, and send those to committing peers. Committing peers then perform validation and committing to the ledger.

All peers make use of certificates issued by membership services.

Clients

Clients are software that makes use of APIs to interact with the Hyperledger Fabric and propose transactions.

Channels

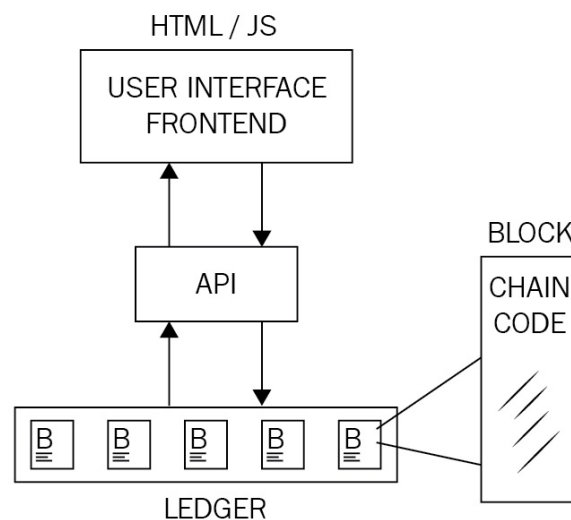
Channels allow the flow of confidential transactions between different parties on the network. They allow using the same blockchain network but with separate blockchains. Channels allow only members of the channel to view the transaction related to them, all other members of the network will not be able to view the transactions.

Transactions

Transaction messages can be divided into two types: deployment transactions and invocation transactions. The former is used to deploy new chaincode to the ledger, and the latter is used to call functions from the smart contract. Transactions can be either public or confidential. Public transactions are open and available to all participants whilst confidential transactions are visible only in a channel open to its participants.

Applications on blockchain

A typical application on Fabric is simply composed of a user interface, usually written in JavaScript/HTML, that interacts with the backend chaincode (smart contract) stored on the ledger via an API layer:



Hyperledger provides various APIs and command-line interfaces to enable interaction with the ledger. These APIs include interfaces for identity, transactions, chaincode, ledger, network, storage, and events.

Consensus in Hyperledger Fabric

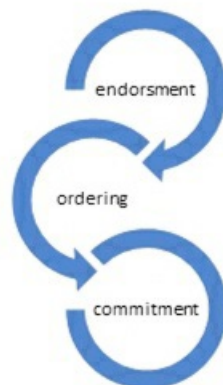
The consensus mechanism in Hyperledger Fabric consists of three steps:

1. Transaction endorsement: This process endorses the transactions by simulating the transaction execution process.

2. Ordering: This is a service provided by the cluster of orderers which takes endorsed transactions and decide on a sequence in which the transactions will be written to the ledger.

3. Validation and commitment: This process is executed by committing peers which first validates the transactions received from the orderers and then commit that transaction to the ledger.

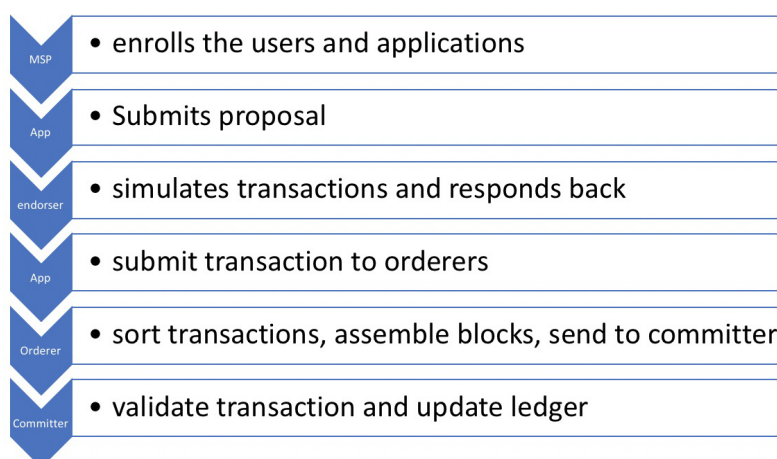
These steps are shown in the following flowchart:



The transaction life cycle in Hyperledger Fabric

There are several steps that are involved in a transaction flow in Hyperledger Fabric. These steps are shown in the following diagram below

A quick summary of the process can be visualized in the following diagram



The steps are described below in detail:

1. Transaction proposal by clients. This is the first step where a transaction is proposed by the clients and sent to endorsing peers on the distributed ledger



network. All clients need to be enrolled via membership services before they can propose transactions.

2. The transaction is simulated by endorsers which generates a read-write (RW) set. This is achieved by executing the chaincode but instead of updating the ledger, only a read-write set depicting any reads or updates to the ledger is created.
3. The endorsed transaction is sent back to the application.
4. Submission of endorsed transactions and read-write (RW) sets to the ordering service by the application.
5. The ordering service assembles all endorsed transactions and read-write sets in order into a block, and sorts them by channel ID.
6. Ordering service broadcasts the assembled block to all committing peers.
7. Committing peers validate the transactions.
8. Committing peers update the ledger.
9. Finally, notification of success or failure of the transaction by committing peers is sent back to the clients/applications.

Sawtooth Lake

Sawtooth Lake can run in both permissioned and non-permissioned modes. It is a distributed ledger that proposes two novel concepts: the first is the introduction of a new consensus algorithm called Proof of Elapsed Time (PoET); and the second is the idea of transaction families.

A description of these novel proposals is given in the following sections.

PoET

PoET is a novel consensus algorithm that allows a node to be selected randomly based on the time that the node has waited before proposing a block. This concept is in contrast to other leader election and lottery-based proof of work algorithms, such as the PoW used in Bitcoin where an enormous amount of electricity and computer resources are used in order to be elected as a block proposer; for example in the case of Bitcoin. PoET is a type of Proof of Work algorithm but, instead of spending computer resources, it uses a trusted computing model to provide a



mechanism to fulfill the Proof of Work requirements. PoET makes use of Intel's SGX architecture (Software Guard Extensions) to provide a trusted execution environment (TEE) to ensure randomness and cryptographic security of the process.

It should be noted that the current implementation of Sawtooth Lake does not require real hardware SGX-based TEE, as it is simulated for experimental purposes only and as such should not be used in production environments. The fundamental idea in PoET is to provide a mechanism of leader election by waiting randomly to be elected as a leader for proposing new transactions.

PoET, however, has a limitation which has been highlighted by Ittay Eyal. This limitation is called the stale chips problem.

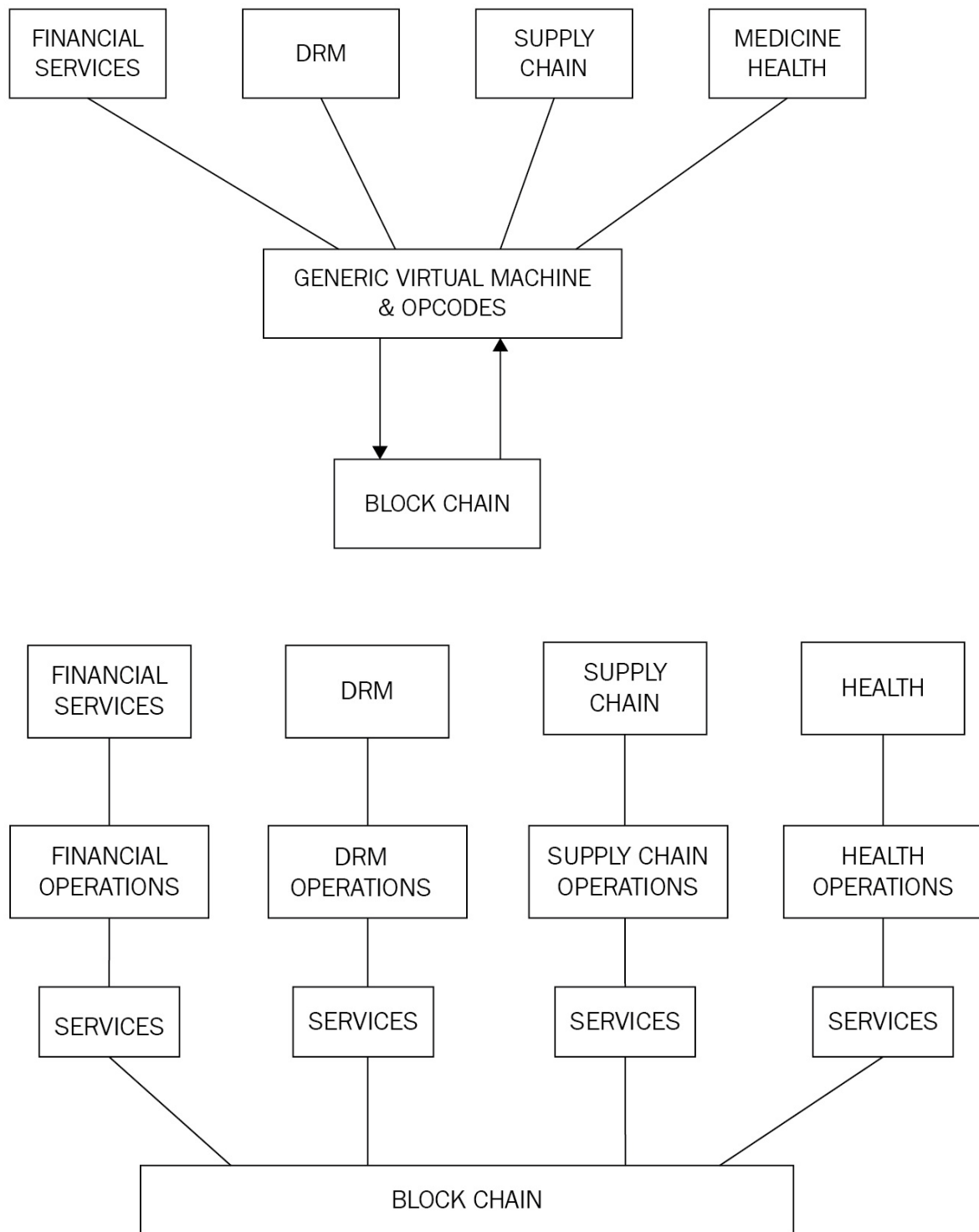
Transaction families

A traditional smart contract paradigm provides a solution that is based on a general-purpose instruction set for

all domains. For example, in the case of Ethereum, a set of opcodes has been developed for the EVM that can be used to build smart contracts to address any type of requirements for any industry.

While this model has its merits, it is becoming clear that this approach is not very secure as it provides a single interface into the ledger with a powerful and expressive language, which potentially offers a larger attack surface for malicious code. This complexity and generic virtual machine paradigm have resulted in several vulnerabilities that were found and exploited recently by hackers. A recent example is the DAO hack and further Denial of Services (DoS) attacks that exploited limitations in some EVM opcodes.

A model shown in the following figure describes the traditional smart contract model, where a generic virtual machine has been used to provide the interface into the blockchain for all domains:



The Sawtooth (transaction families) smart contract paradigm

Consensus in Sawtooth

Sawtooth has two types of consensus mechanisms based on the choice of network. PoET, as discussed previously, is a trusted executed environment-based



lottery function that elects a leader randomly based on the time a node has waited for block proposal.

There is another consensus type called quorum voting, which is an adaptation of consensus protocols built by Ripple and Stellar. This consensus algorithm allows instant transaction finality, which is usually desirable in permissioned networks.

CORDA

Corda is not a blockchain by definition because it does not contain blocks of bundled transactions, but it falls under the category of distributed ledgers. It provides all benefits that a blockchain can. Traditional blockchain solutions, as discussed before, have the concept of transactions that are bundled together in a block and each block is linked back cryptographically to its parent block, which provides an immutable record of transactions. This is not the case with Corda.

Corda has been designed entirely from scratch with a new model for providing all blockchain benefits, but without a traditional blockchain. It has been developed purely for the financial industry to solve issues arising from the fact that each organization manages their own ledgers and thus have their own view of truth, which leads to contradictions and operational risk. Moreover, data is also duplicated at each organization, which results in an increased cost of managing individual infrastructures and complexity. These are the types of problems within the financial industry that Corda aims to resolve by building a decentralized database platform.

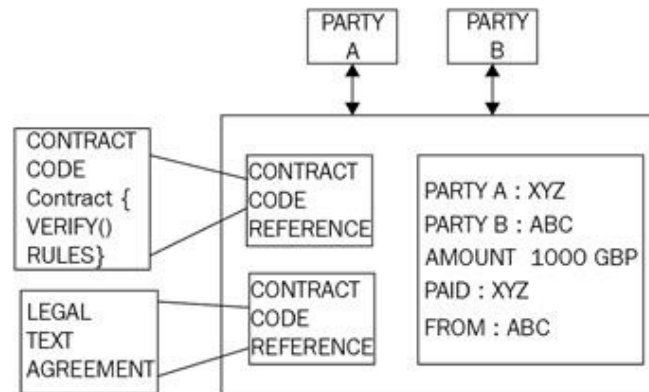
Architecture

The main components of the Corda platform include state objects, contract code, legal prose, transactions, consensus, and flows. We will now explore them in more detail.

State objects

State objects represent the smallest unit of data that represent a financial agreement. They are created or deleted as a result of a transaction execution. They refer to contract code and legal prose. Legal prose is optional and provides legal binding to the contract. However, contract code is mandatory in order to manage the state of the object. It is required in order to provide a state transition mechanism for the node according to the business logic defined in the contract code. State objects contain a data structure that represents the current state of the object. A state object can be either current (live) or historic (no longer valid).

For example, in the following diagram, a state object represents the current state of the object. In this case, it is a simple mock agreement between Party A and Party B where Party ABC has paid Party XYZ 1,000 GBP. This represents the current state of the object; however, the referred contract code can change the state via transactions. State objects can be thought of as a state machine, which are consumed by transactions in order to create updated state objects.



Transactions

Transactions are used to perform transitions between different states. For example, the state object shown in the preceding diagram is created as a result of a transaction. Corda uses a Bitcoin-style UTXO-based model for its transaction processing. The concept of state transition by transactions is same as in Bitcoin. Similar to Bitcoin, transactions can have none, single, or multiple inputs, and single or multiple outputs. All transactions are digitally signed.

Moreover, Corda has no concept of mining because it does not use blocks to arrange transactions in a blockchain. Instead, notary services are used in order to provide temporal ordering of transactions. In Corda, new transaction types can be developed using JVM bytecode, which makes it very flexible and powerful.

Consensus

The consensus model in Corda is quite simple and is based on notary services that are discussed in a later section of this chapter. The general idea is that the transactions are evaluated for their uniqueness by the notary service and, if they are unique (that is, unique transaction inputs), they are signed by consensus services as valid. There can be single or multiple clustered notary services running on a Corda network. Various consensus algorithms like PBFT or Raft can be used by notaries to reach consensus.

There are two main concepts regarding consensus in Corda: consensus over state validity and consensus over state uniqueness. The first concept is

concerned with the validation of the transaction, ensuring that all required signatures are available and states are appropriate. The second concept is a means to detect double-spend attacks and ensures that a transaction has not already been spent and is unique.

Flows

Flows in Corda are a novel idea that allows the development of decentralized workflows. All communication on the Corda network is handled by these flows. These are transaction-building protocols that can be used to define any financial flow of any complexity using code. Flows run as an asynchronous state machine and they interact with other nodes and users. During the execution, they can be suspended or resumed as required.

Components

The Corda network has multiple components. All these components are described in the upcoming sections.

Nodes

Nodes in a Corda network operated under a trust-less model and run by different organizations. Nodes run as part of an authenticated peer-to-peer network. Nodes communicate directly with each other using the Advanced Message Queuing Protocol (AMQP), which is an approved international standard (ISO/IEC 19464) and ensures that messages across different nodes are transferred safely and securely. AMQP works over Transport Layer Security (TLS) in Corda, thus ensuring privacy and integrity of data communicated between nodes.

Nodes also make use of a local relational database for storage. Messages on the network are encoded in a compact binary format. They are delivered and managed by using the Apache Artemis message broker (Active MQ). A node can serve as a network map service, notary, Oracle, or a regular node. The following diagram shows a high-level view of two nodes communicating with each other:



Two nodes communicating in a Corda network



In the preceding diagram, Node 1 is communicating with Node 2 over a TLS communication channel using the AMQP protocol, and the nodes have a local relational database for storage.

Network map service

This service is used to provide a network map in the form of a document of all nodes on the network. This service publishes IP addresses, identity certificates, and a list of services offered by nodes. All nodes announce their presence by registering to this service when they first startup, and when a connection request is received by a node, the presence of the requesting node is checked on the network map first. Put another way, this service resolves the identities of the participants to physical nodes.

Notary service

In a traditional blockchain, mining is used to ascertain the order of blocks that contain transactions. In Corda, notary services are used to provide transaction ordering and timestamping services. There can be multiple notaries in a network and they are identified by composite public keys. Notaries can use different consensus algorithms like BFT or Raft depending on the requirements of the applications. Notary services sign the transactions to indicate validity and finality of the transaction which is then persisted to the database.

Transactions

Transactions in a Corda network are never transmitted globally but in a semi-private network. They are shared only between a subset of participants who are related to the transaction. This is in contrast to traditional blockchain solutions like Ethereum and Bitcoin, where all transactions are broadcasted to the entire network globally. Transactions are digitally signed and either consume state(s) or create new state(s).

Transactions on a Corda network are composed of the following elements:

Input references: This is a reference to the states the transaction is going to consume and use as an input.

Output states: These are new states created by the transaction.

Attachments: This is a list of hashes of attached ZIP files. ZIP files can contain code and other relevant documentation related to the transaction. Files



themselves are not made part of the transaction, instead, they are transferred and stored separately.

Commands: A command represents the information about the intended operation of the transaction as a parameter to the contract. Each command has a list of public keys, which represents all parties that are required to sign a transaction.

Signatures: This represents the signature required by the transaction. The total number of signatures required is directly proportional to the number of public keys for commands.

Type: There are two types of transactions namely, normal or notary changing. Notary changing transactions are used for reassigning a notary for a state.

Timestamp: This field represents a bracket of time during which the transaction has taken place. These are verified and enforced by notary services. Also, it is expected that if strict timings are required, which is desirable in many financial services scenarios, notaries should be synced with an atomic clock.

Summaries: This is a text description that describes the operations of the transaction.

CorDapp

The core model of Corda consists of state objects, transactions, and transaction protocols, which when combined with contract code, APIs, wallet plugins, and user interface components results in constructing a Corda distributed application (CorDapp).

Smart contracts in Corda are written using Kotlin or Java. The code is targeted for JVM.

JVM has been modified slightly in order to achieve deterministic results of execution of JVM bytecode. There are three main components in a Corda smart contract as follows:

- Executable code that defines the validation logic to validate changes to the state objects.
- State objects represent the current state of a contract and either can be consumed by a transaction or produced (created) by a transaction.
- Commands are used to describe the operational and verification data that defines how a transaction can be verified.



RV Institute of Technology and Management®